Thanit Mitparien

August 11, 2026

Foundations Of Python Programming

Assignment: 07

https://github.com/ThanitM/IntroToProg-Python-Mod07

Course Registration Program with Data Classes

Introduction

This document outlines the process of completing Assignment 07 for the Foundation of Python Programming course. Building upon Assignment 06, where a Python script was developed to input student information, display records, and save data to a JavaScript Object Notation (JSON) file using Python dictionaries and error handling, functions, classes and the Separation of Concerns (SoC) design pattern. This assignment further enhances the script by introducing **a data classes, constructor, getter method, setter method, class inheritance and overridden method**.

As the starter script provides the foundational structure and majority of the base code, this document will skip those pre-existing implementations and focus specifically on the updates made to fulfill the assignment objectives — explaining how data classes, constructors, class inheritance and overridden method are applied to refactor and improve the script.

High-Level Process

There are total of seven (7) steps to complete this assignment:

1.) Create Data Class Named “Person”
    a. Add “first_name” and “last_name” Properties to The Constructor
    b. Create A Getter and Setter for the “first_name” Property
    c. Create A Getter and Setter for the “last_name” Property
    d. Override the “__str__()” Method to Return Person Data
2.) Create Person Class’s Inherit Data Class Named “Student”
    a. Call to the “Person” Constructor to Initialize “first_name” and “last_name”
    b. Add “course_name” Property to the Constructor
    c. Create A Getter and Setter for the “course_name”
    d. Override the “__str__()” Method to Return Student Data
3.) Convert Dictionary Data to Student Data in “read_data_from_file” Method

4.) Convert Student Objects into Dictionaries in “write_data_to_file” Method
5.) Update Print Statement to Output Student Object Data Instead of Dictionary Data in “output_student_and_course_names” Method
6.) Using A Student Objects Data Instead of a Dictionary Objects in “input_student_data” method
7.) Run and Test the Script

1. Create Data Class Named *“Person”*

As part of Assignment 07’s Acceptance Criteria, the program will need to include a class named *“Person”* where it will be used as a data class to organize data and represent information about the object. In this case, “Person” data class will represent information about people. Inside this class, there will be two (2) variables or class properties – first_name and last_name which will store the names of the individuals. Later in the program, this program will initialize objects using this class. Figure 1 shows how class *“Person”* is defined along with class document strings.

```
class Person:  1 usage
    '''
    A class representing person data object

    Properties:
        first_name (str): First name of the person
        last_name (str): Last name of the person

    ChangeLog: (Who, When, What)
    Thanit Mitparien, 08/10/2026, Created a class
    '''

```

Figure 1: Person Class

a. Add *“first_name”* and *“last_name”* Properties to The Constructor

The constructor is the special method that will be automatically called to initialize class’s properties when the class object is being created. For the “Person” class, there are two (2) properties – “first_name” and “last_name”. The constructor can be implemented by overriding “__init__()” method. Figure 2 shows the implementation of Person class.

```
    def __init__(self, first_name, last_name):
        '''
        This is a class constructor used to initialize a person object
        :param first_name:
        :param last_name:

        ChangeLog: (Who, When, What)
        Thanit Mitparien, 08/10/2026, Created a constructor
        '''
        self.__first_name = first_name
        self.__last_name = last_name
```

Figure 2: Person Class’s Constructor Method

b. Create A Getter and Setter for the "*first_name*" Property

To enforce the idea of the encapsulation idea of the programing and prevent users from inappropriately make changes to the class properties. This assignment has specified that there will need to be a getter and a setter method for the "first_name". To implement the getter method, we will need to first call "@property" decorator then define a method as show in figure 3, line 59-60. To implement the setter method, we will need to call "@<property_name>.setter" in our case, it would be "@first_name.setter" as shows in figure 3, line 71-72.

```
59      @property  5 usages (2 dynamic)
60      def first_name(self):
61          '''
62          This function returns the first name of the person
63          :return: first_name (str)
64
65          ChangeLog (who, When, What)
66          Thanit Mitparien, 08/10/2026, Created a getter function for
67          a class property - "first_name: str"
68          '''
69          return self.__first_name.title()
70
71      @first_name.setter  3 usages (2 dynamic)
72      def first_name(self, value : {isalpha, __eq__} ):
73          '''
74          This function sets the first name of the person class object
75
76          :param value: first_name (str)
77          :return: None
78
79          ChangeLog: (Who, When, What)
80          Thanit Mitparien, 08/10/2026, Created a setter function for
81          a class property - "first_name: str"
82          '''
83          if value.isalpha() or value == "":
84              self.__first_name = value
85          else:
86              raise ValueError("The first name should not contain numbers!")
```

Figure 3: Getter and Setter Methods for "first_name" Property

c. Create A Getter and Setter for the "*last_name*" Property

Similarly to the "first_name" property, we will also need to implement getter and setter method for "last_name" property also. Figure 4, line 88 – 116 shows how these methods are implemented for this property.

```python
    @property  5 usages (2 dynamic)
    def last_name(self):
        '''
        This function returns the last name of the person class object

        :return: last_name: str

        ChangeLog: (Who, When, What)
        Thanit Mitparien, 08/10/2026, Created a getter function for
        a class property - "last_name: str"
        '''
        return self.__last_name.title()

    @last_name.setter  3 usages (2 dynamic)
    def last_name(self, value : {isalpha, __eq__} ):
        '''
        This function sets the last name of the person class object

        :param value: last_name(str)
        :return: None

        ChangeLog: (Who, When, What)
        Thanit Mitparien, 08/10/2026, Created a setter function for
        a class property - "last_name: str"
        '''
        if value.isalpha or value == "":
            self.__last_name = value
        else:
            raise ValueError("The last name should not contain numbers!")
```

Figure 4: Getter and Setter Method for "last_name" Property

d. Override "*__str__()*" Method to Return Person Data

To satisfy another acceptance criteria of the Assignment 07, we will have to override a "__str__()" method in a Person class to display "first_name" and "last_name" of a person data. Figure 5 shows how the "__str__()" was overridden and implemented.

```python
    def __str__(self):
        '''
        This function returns the string representation of the person class object

        :return: "first_name, last_name"

        ChangeLog: (Who, When, What)
        Thanit Mitparien, 08/10/2026, Created a string dunder function for a Person class
        '''
        return f"{self.first_name},{self.last_name}"
```

Figure 5: Person Class's __str__() Overridden Method

2. Create Person Class's Inherit Data Class Named *"Student"*

The "Student" class is a data class that holds student's information and it's a child class of the "Person" class. To inherit a class in the Python, we define the name of a child class follow by the parent class in the parenthesis. Figure 6 shows how "Student" data class is defined and how it inherited from the "Person" class.

```
136  class Student(Person):  4+ usages
137      '''
138       A class representing Student data object which is an inherited class from
139       the Person class data object
140
141      Properties:
142          first_name (str): First name of the person
143          last_name (str): Last name of the person
144          course_name (str): Registered course name
145
146      ChangeLog: (Who, When, What)
147      Thanit Mitparien, 08/10/2026, Created a class
148      '''
```

Figure 6: "Student" Class Definition

a. Call to the "Person" Constructor to Initialize "first_name" and "last_name"

Since the "Student" class is a child class of the "Person" class, the program can inherit properties of the "Person" class. In this case, "Student" class will inherit "first_name" and "last_name" so the object can initialize these properties using "Person" class. To accomplish this, the program will need to use call "super()" function following by ".__init__(first_name, last_name)" in Student's constructor. Figure 7 shows how Student's constructor was defined.

```
150      def __init__(self, first_name: str = '', last_name: str = '', course_name: str = ''):
151          '''
152          This constructor initializes Student class properties
153
154          :param first_name:
155          :param last_name:
156          :param course_name:
157
158          ChangeLog: (Who, When, What)
159          Thanit Mitparien, 08/10/2026, Created a constructor
160          '''
161          # Initialize first_name and last_name with Person Class Constructor
162          super().__init__(first_name=first_name, last_name=last_name)
163          self.__course_name = course_name
164
```

Figure 7: "Student" Class Constructor Definition

b. Add “course_name” Property to the Constructor

“Course_name” property is a specific data to the “Student” class so the constructor will be defining and initializing it as how the “first_name” and “last_name” were defined in “Person” class. Figure 7, line 163 shows how “course_name” property was initialized in the constructor.

c. Create A Getter and Setter for the “course_name”

To define a getter and setter for the “course_name” property for the “Student” class, the same implementation to the “first_name” and “last_name” in “Person” class applies here. Figure 8 shows how “course_name” getter and setter methods are defined in the program.

```
165     @property  4 usages (2 dynamic)
166     def course_name(self):
167         '''
168         This getter functions return course_name property
169
170         :return: Course_name (str)
171
172         ChangeLog: (Who, When, What)
173         Thanit Mitparien, 08/10/2026, Created a getter function
174         '''
175         return self.__course_name.title()
176
177     @course_name.setter  2 usages (2 dynamic)
178     def course_name(self, value : {isalpha, __eq__} ):
179         '''
180         This setter function sets the course_name property
181
182         :param value: course_name (str)
183         :return: None
184
185         ChangeLog: (Who, When, What)
186         Thanit Mitparien, 08/10/2026, Created a setter function
187         '''
188         if value.isalpha or value == "":
189             self.__course_name = value
190         else:
191             raise ValueError("The course name should not contain numbers!")
192
```

Figure 8: “course_name” Getter and Setter Method of “Student” Class

d. Override the “__str__()” Method to Return Student Data

The override “__str__()” method to return Student data is also the same as the override “__str__()” method in “Person” class. However, the Student class will return “first_name”, “last_name” and “course_name”. Figure 9: shows how “__str__()” method is defined in “Student” class.

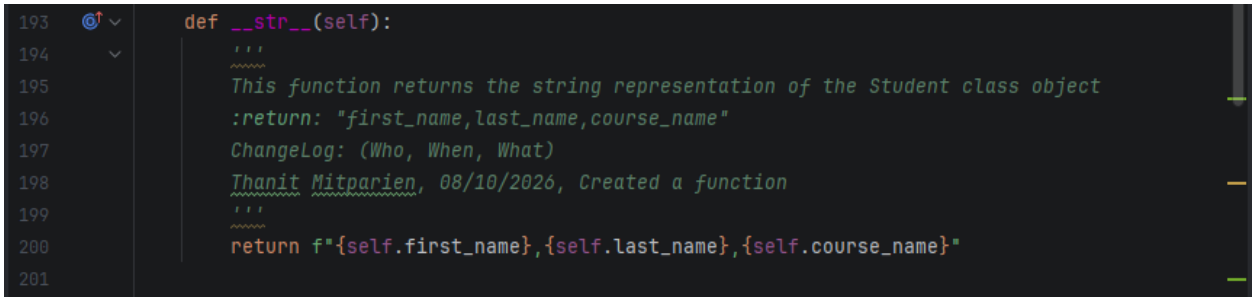

Figure 9: "__str__()" Override Method of "Student" Class.

3. Convert Dictionary Data to Student Data in "read_data_from_file" Method

Now that the program has "Student" data class, the "read_data_from_file" method in "FileProcessor" class will need to be updated as well to convert student data from dictionary data type to "Student" object. To accomplished this, we will be defined "student_object: Student" variable which is an object of "Student" class then initialize the object with student information from "Enrollments.json" file using a constructor from "Student" class. Figure 10, line 233 – 237 shows how the program accomplished this update.

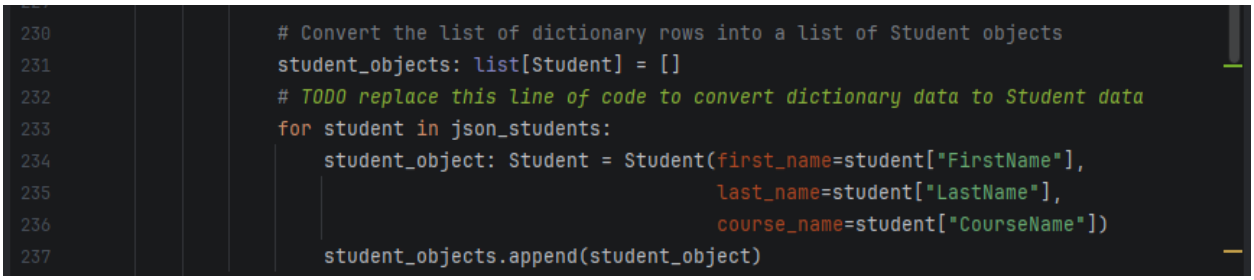

Figure 10: "Student" Class Object Conversion

4. Convert Student Objects into Dictionaries in "write_data_to_file" Method

The "write_data_to_file" method in "FileProcessor" class is another method that need to be updated due to "Student" class object. To read an information from an object so it can be written to a file, the program will need to define as the following: <Object Name>.<Object's property>. Figure 11, line 265 - 270 shows how the program achieves the conversion.

```
263         try:
264             # TODO Add code to convert Student objects into dictionaries (Done)
265             list_of_dictionary_data: list = []
266             for student in student_data:
267                 student_json: dict = {"FirstName": student.first_name,
268                                       "LastName": student.last_name,
269                                       "CourseName": student.course_name}
270                 list_of_dictionary_data.append(student_json)
```

Figure 11: The "write_data_to_file" method conversion.

5. Update Print Statement to Output Student Object Data Instead of Dictionary Data in "output_student_and_course_names" Method

Similar to the conversion in the previous sub-section, "output_student_and_course_names" method in "IO" class also has to be updated with the object dot operation to get object's property/data. Figure 12, line 361 - 364 shows how the conversion to access Students object data instead of dictionary data looks like.

```
358         print("-" * 50)
359         for student in student_data:
360 
361             # TODO Add code to access Student object data instead of dictionary data
362             print(f'Student {student.first_name} '
363                   f'{student.last_name} is enrolled in {student.course_name}')
364 
365         print("-" * 50)
366 
```

Figure 12: The "output_student_and_course_names" Method Conversion

6. Using A Student Objects Data Instead of a Dictionary Objects in "input_student_data" method

Similar to "read_data_from_file" method in "FileProcessor" class, the "input_student_data" method in "IO" class also needs to be updated to use "Student" object data instead of a dictionary data. Figure 13 shows you how to accomplish this.

```
389             # TODO Replace this code to use a Student objects instead of a dictionary objects
390             student = Student(first_name=student_first_name,
391                               last_name=student_last_name,
392                               course_name=course_name)
393 
```

Figure 13: "input_student_data" Conversion

7. Run and Test

This section will include the script run to test whether the implementation produces the result as we expected. The acceptance criteria have provided seven (7) total test cases for us to run. Figure 14 shows us the results for single user input and Figure 16 show us the results for multiple user inputs while running on PyCharm. Figure 15 and 17 show the "*Enrollments.json*" file after writing additional data to it on PyCharm. Figure 18 shows us the results for single user input and Figure 20 show us the results for multiple user inputs while running on Windows Console. Figure 19 and 21 show the "*Enrollments.json*" file after writing additional data to it on Windows Console.

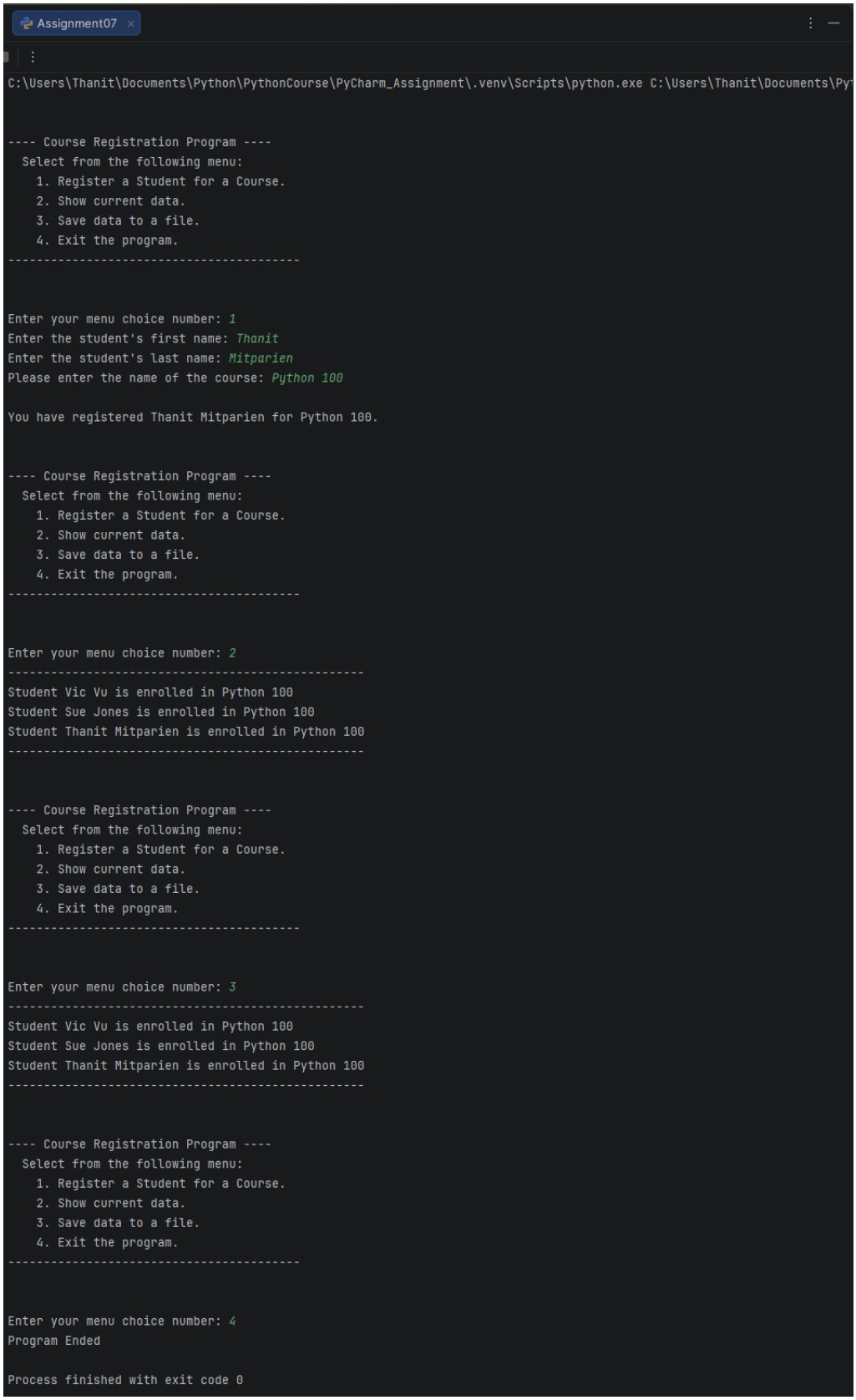

Figure 14: Single User's input Run/Test on PyCharm

Figure 15: Single User's Input Enrollments.json File Update on PyCharm Run

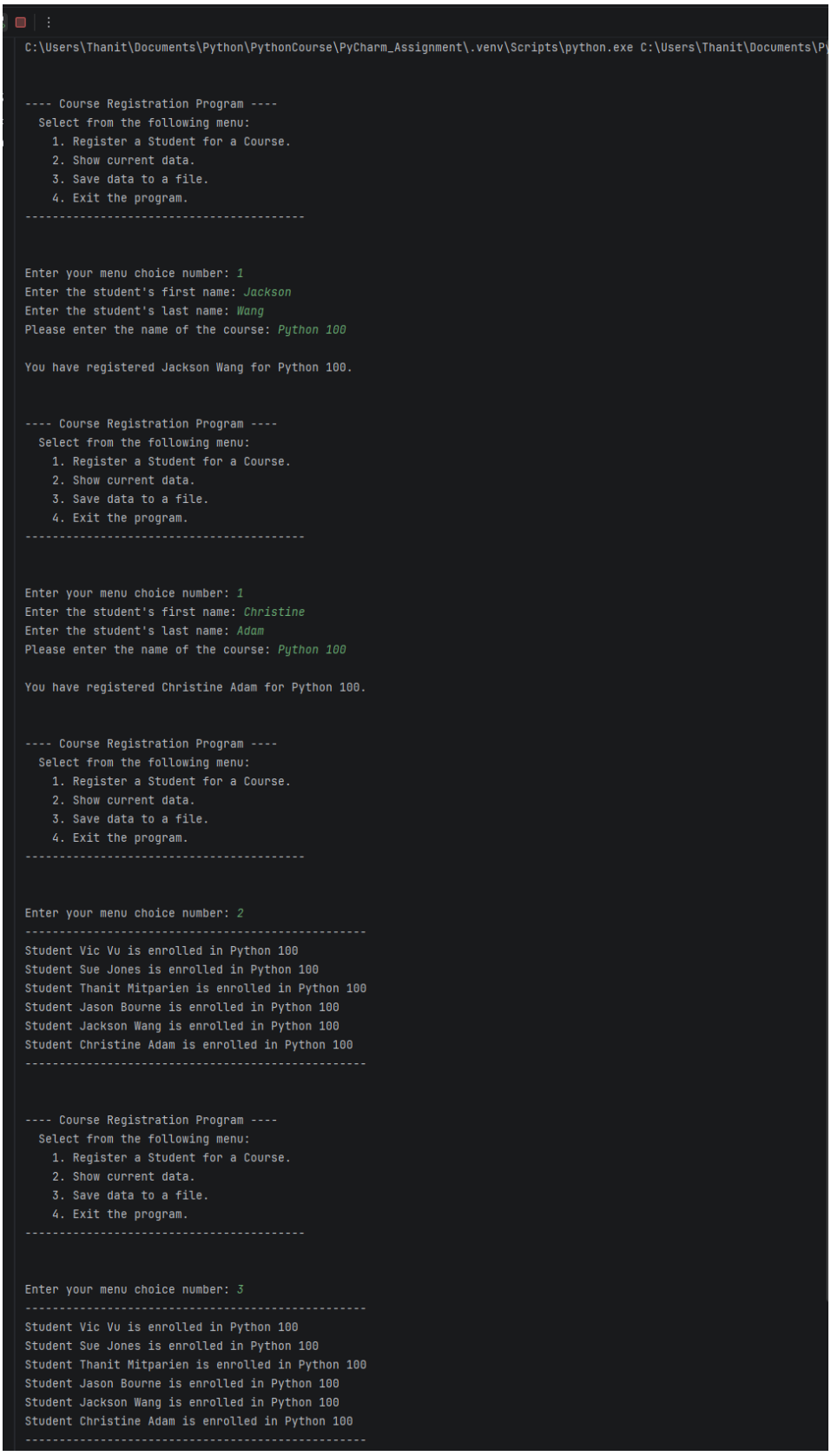

Figure 16: Multiple User's inputs Run/Test on PyCharm

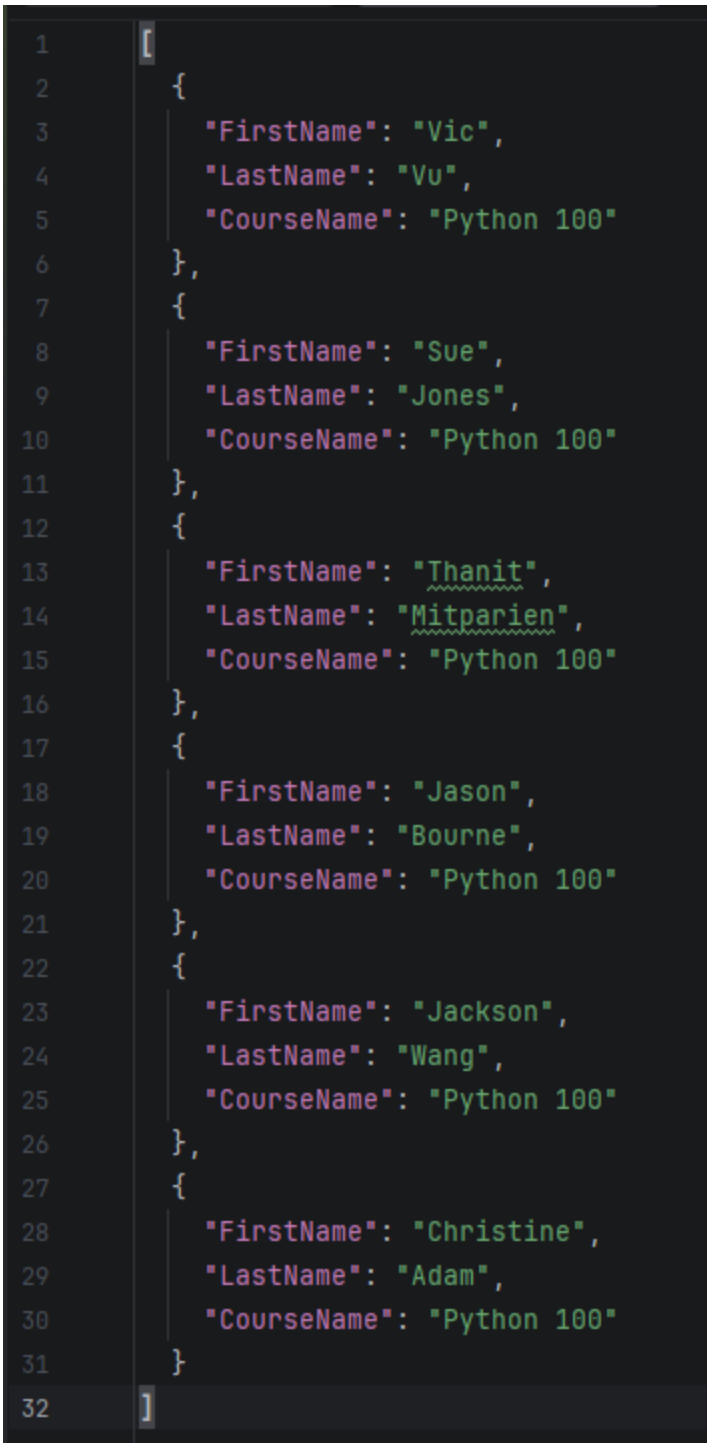

Figure 17: Multiple User's Input Enrollments.json File Update on PyCharm Run

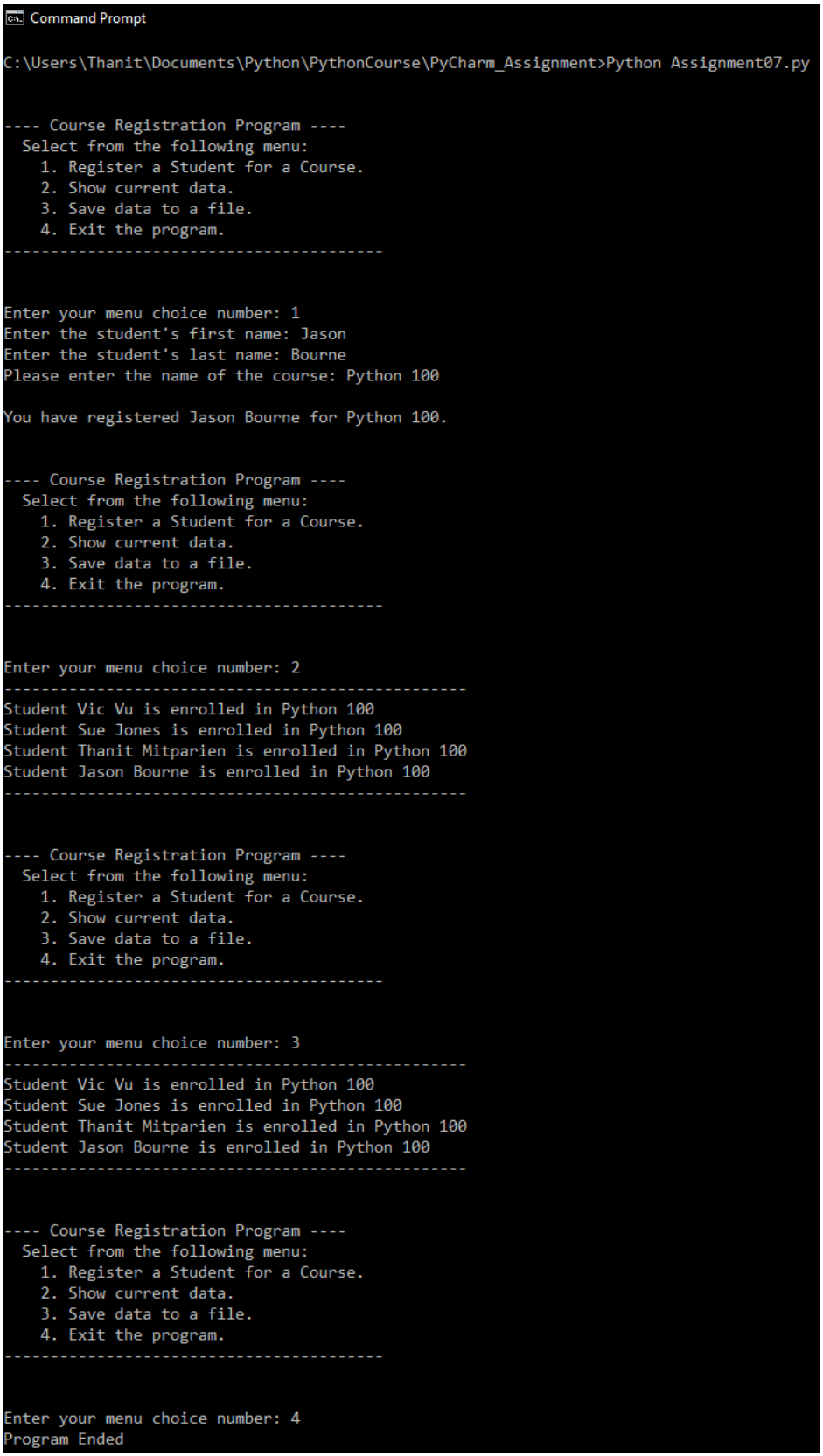

Figure 18: Single User's input Run/Test on Windows Console

Figure 19: Single User's Input Enrollments.json File Update on Windows Console Run

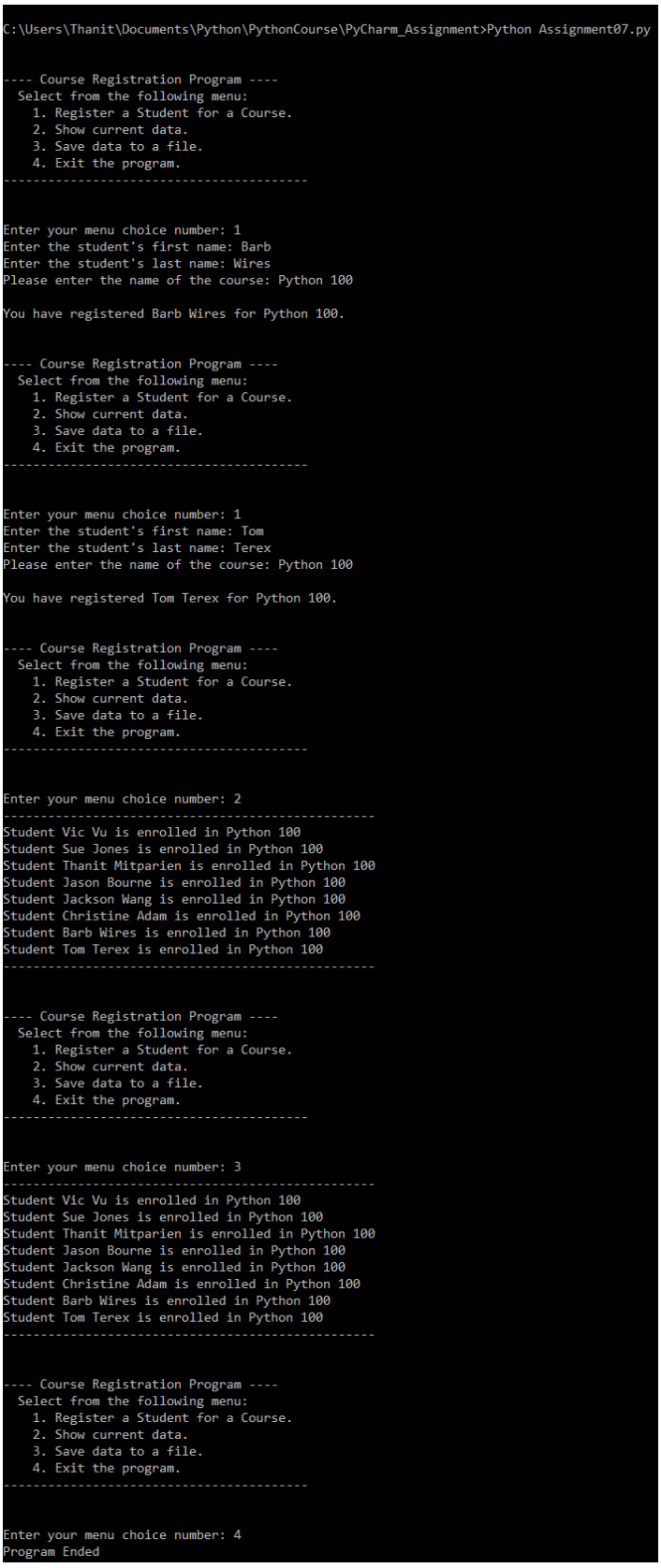

Figure 20: Multiple User's inputs Run/Test on Windows Console

Figure 21: Multiple User's Input Enrollments.json File Update on Windows Console Run

Summary

This report documents the completion of Assignment 07 for the Foundation of Python Programming course. The assignment extends the Python script developed in Assignment 06, which allowed users to input student information, display records, and save data to a JSON file using Python dictionaries, error handling, functions, classes and the Separation of Concerns (SoC) design pattern.

The key focus of this assignment is the refactoring and enhancement of the existing script through the introduction of **data classes, class constructors, class attributes, properties, class inheritance and overridden methods**. These improvements aim to produce a more structured, readable, and maintainable codebase by organizing logic into distinct and reusable components.

As the starter script provides the base implementation, this document concentrates on the specific updates and modifications made to meet the assignment requirements, detailing how data classes, class constructors, class inheritance and overridden methods were applied to improve the overall script.